

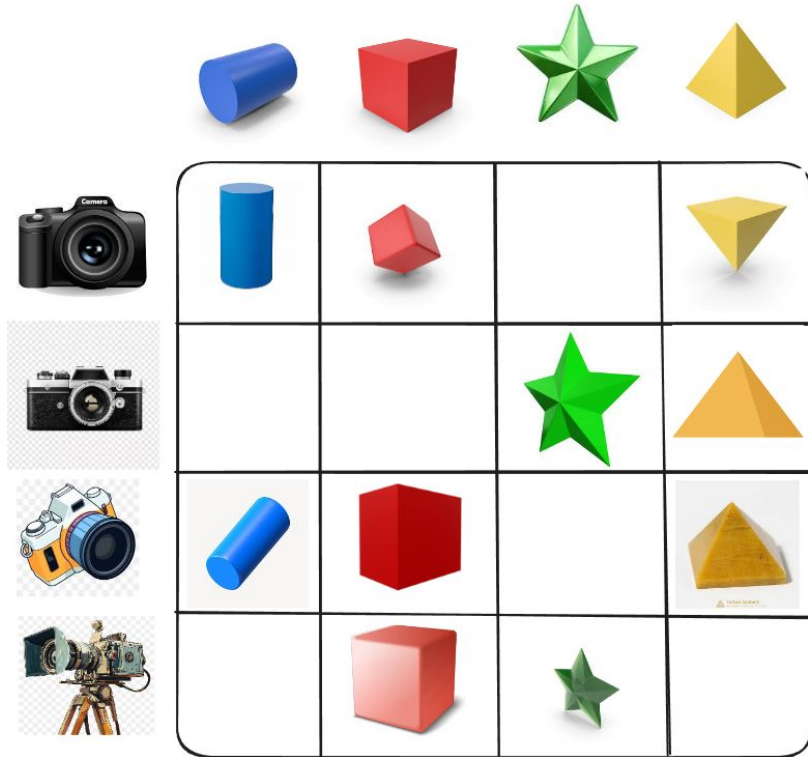
Bundle Adjustment

Why Levenberg-Marquardt?

Why have this tutorial?

- BA is intimidating. I wish someone explained it to me this way
- The optimization part of BA is brushed over/handwavy
- Like how does it work — EXACTLY??
- Our group mostly uses gradient descent, so it's a good exercise to study problems where it can be avoided

Bundle Adjustment: Problem Statement



Data

- 3D ~~objects~~ points
- Camera ~~models~~ parameters
- 2D ~~images~~ projections

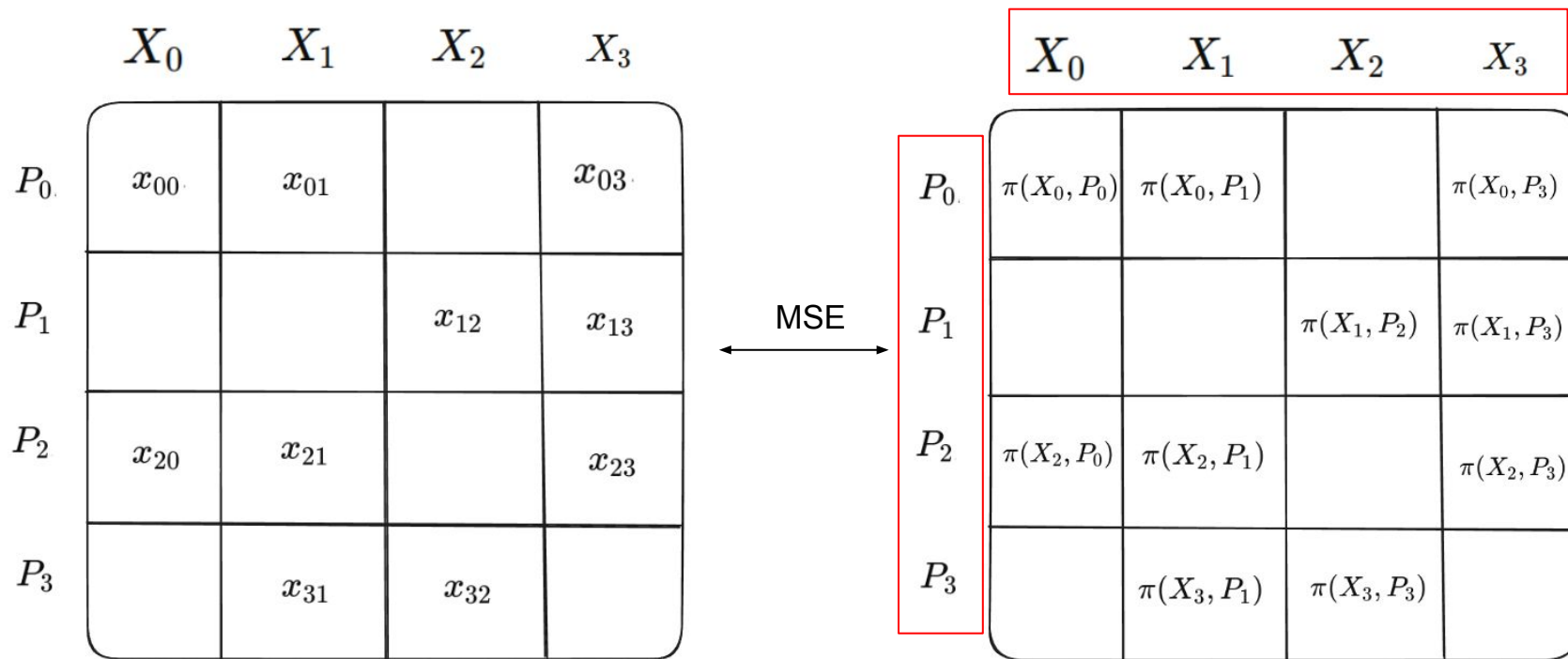
Bundle Adjustment: Problem Statement

	X_0	X_1	X_2	X_3
P_0	x_{00}	x_{01}		x_{03}
P_1			x_{12}	x_{13}
P_2	x_{20}	x_{21}		x_{23}
P_3		x_{31}	x_{32}	

Data

- 3D points $\mathbf{X}_i$
- Camera parameters $\mathbf{P}_i$
- 2D point projections $\mathbf{x}_{ij}$

Bundle Adjustment: Problem Statement



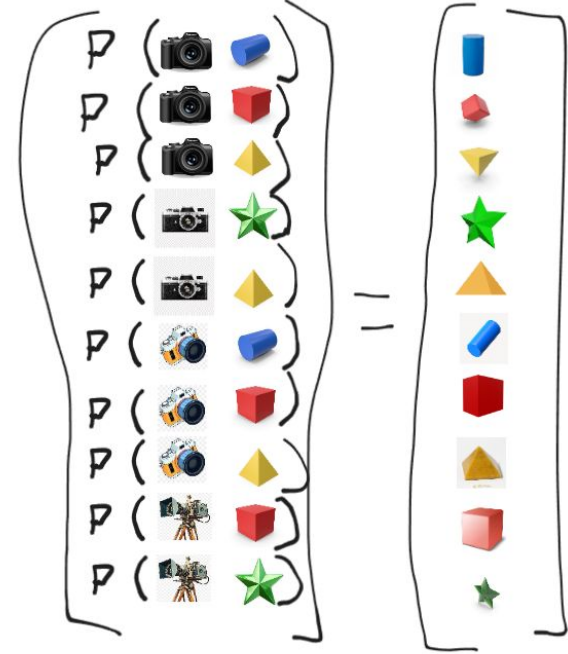
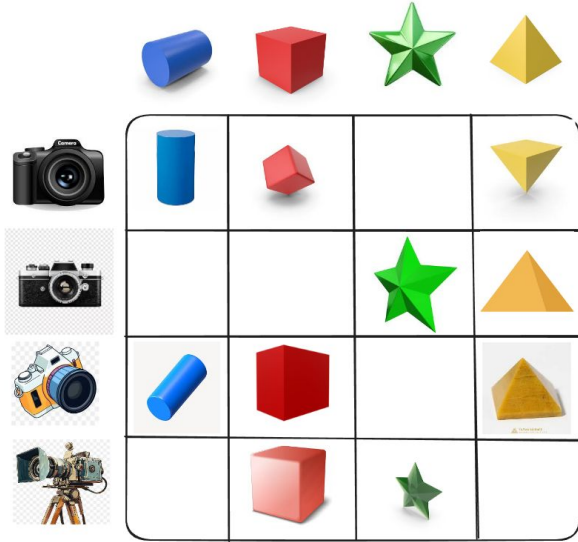
Bundle Adjustment: FAQ

- Q: Where do the camera parameters come from?
A: Manual calibration
- Q: What are they?
A: **For today** it's focal length, distortion parameters and pose
- Q: Where do the points come from?
A: Feature tracking (???)
- Q: Why estimate points and parameters when we already have them?
A: Joint refinement. One camera observes many points, one points is observed in many cameras

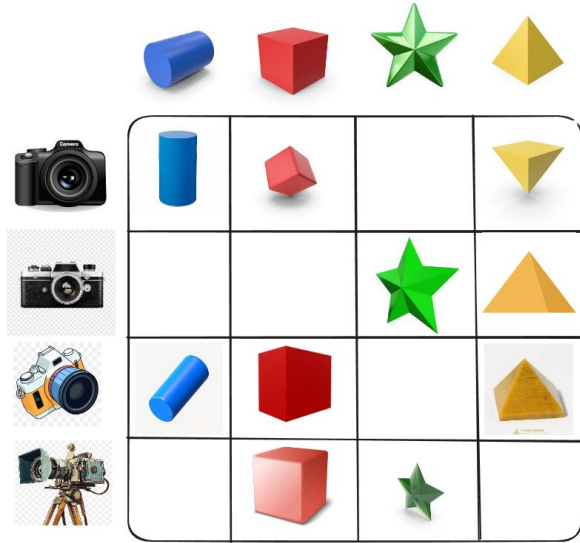
Bundle Adjustment: Problem Statement

$$\min_{\mathbf{P}_j, \mathbf{X}_i} \sum_{i=1}^n \sum_{j=1}^m \|\mathbf{x}_{ij} - \pi(\mathbf{X}_i, \mathbf{P}_j)\|^2$$

Bundle Adjustment: Problem Statement



Bundle Adjustment: Problem Statement



$$\sum \left(\left[\begin{array}{c} P \left(\begin{array}{c} \text{Canon} \\ \text{Nikon} \\ \text{Sony} \\ \text{GoPro} \end{array} \right) \\ \vdots \\ P \left(\begin{array}{c} \text{Canon} \\ \text{Nikon} \\ \text{Sony} \\ \text{GoPro} \end{array} \right) \end{array} \right] - \left[\begin{array}{c} \text{Blue Cylinder} \\ \text{Red Cube} \\ \text{Green Star} \\ \text{Yellow Pyramid} \end{array} \right] \right)^2 = \left[\begin{array}{c} \text{Canon} \\ \text{Nikon} \\ \text{Sony} \\ \text{GoPro} \end{array} \right]$$

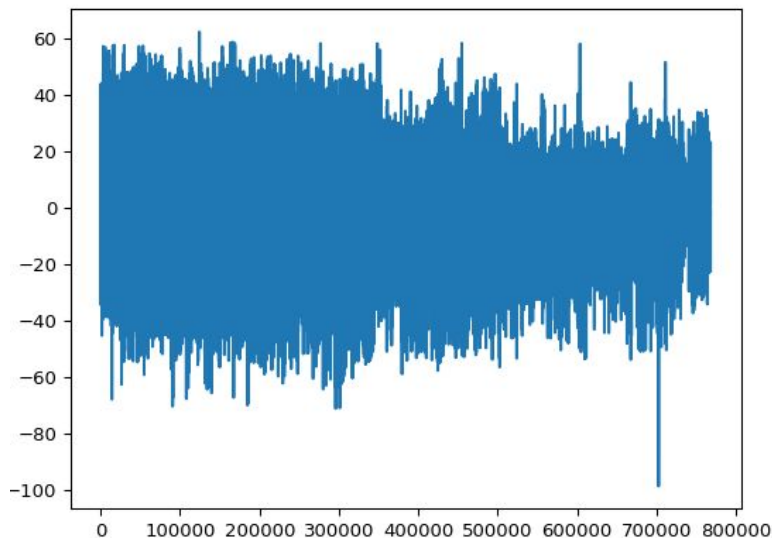
$$\arg \min_x f(x)$$

Bundle Adjustment: Optimization

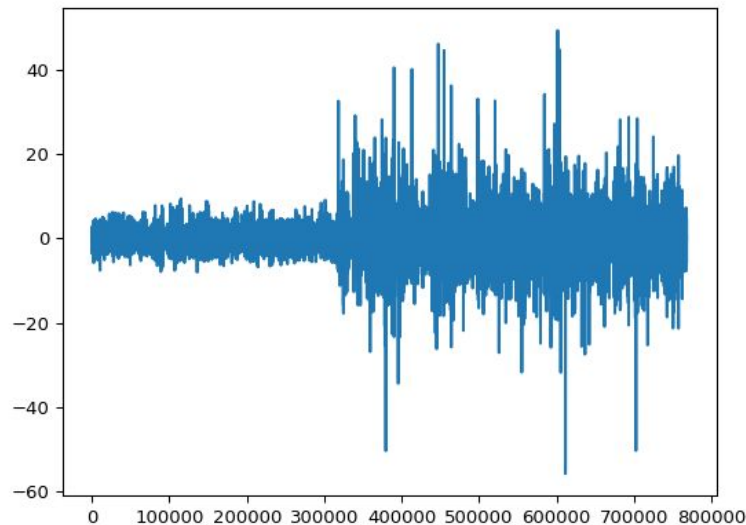


$$\arg \min_x f(x)$$

Bundle Adjustment: Adam Optimization Results

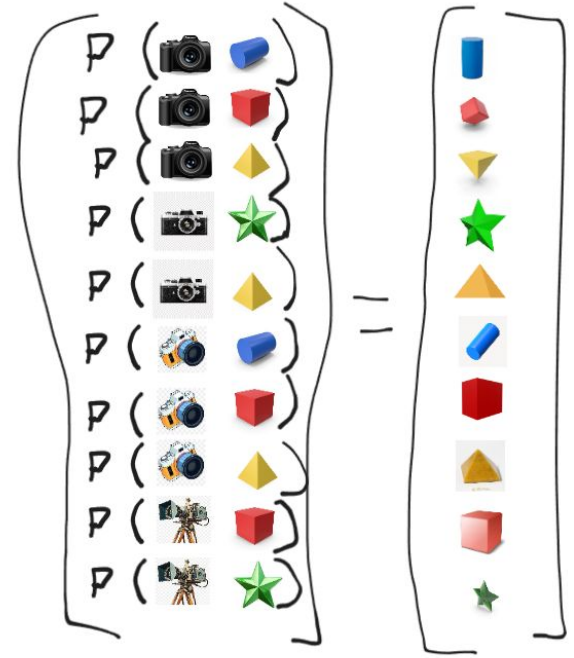
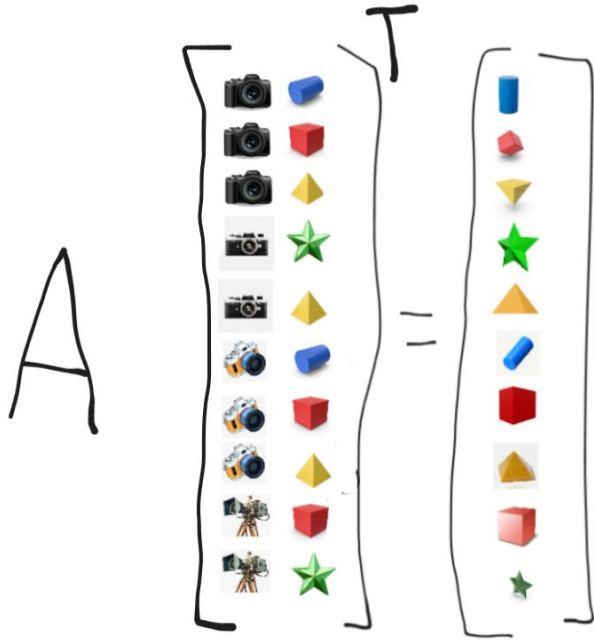


Before
Residuals

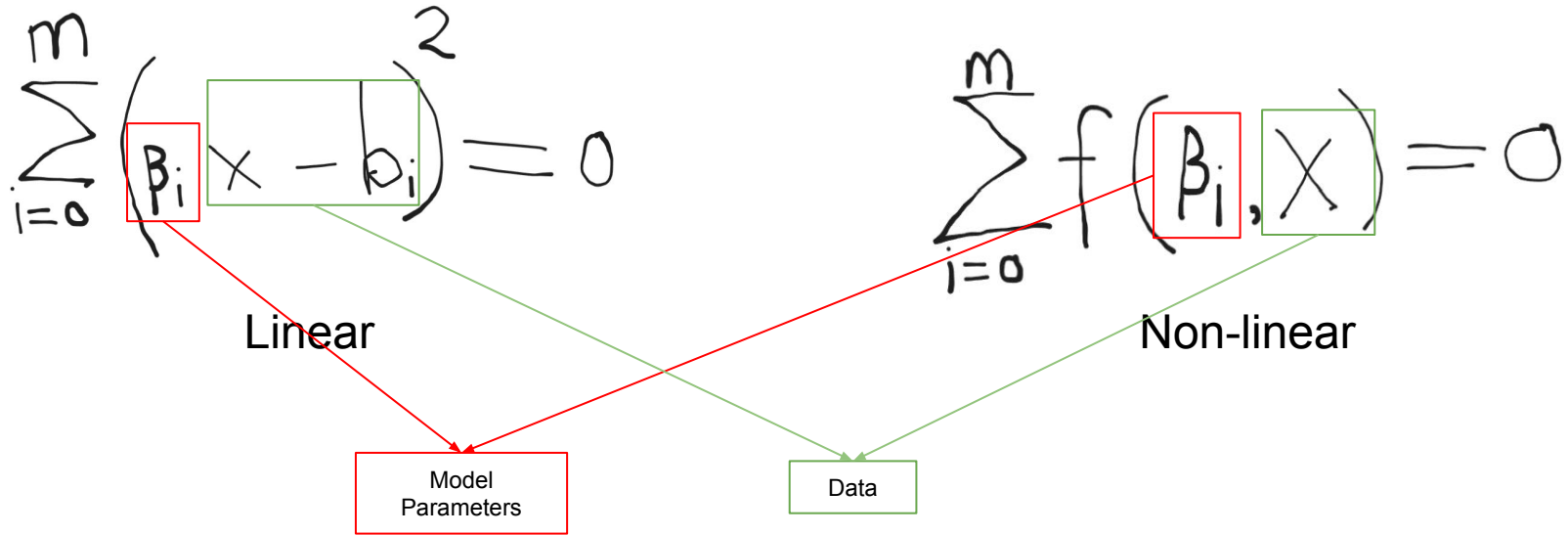


After
Residuals

Bundle Adjustment: Least Squares



Bundle Adjustment: Least Squares Formulation

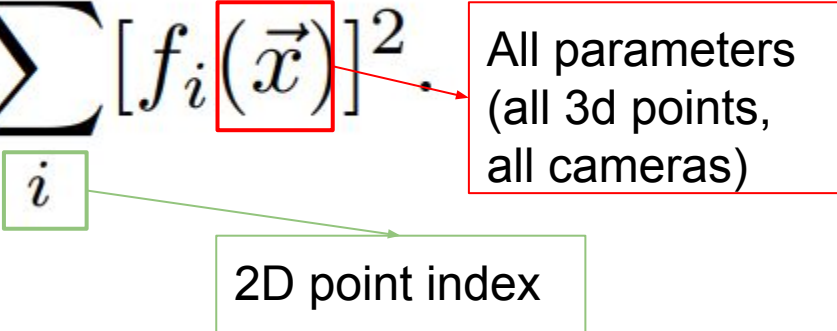


Bundle Adjustment: Least Squares Formulation

$$\sum_{i=0}^m f(\beta, X_i) = \sum_{i=0}^m \left(\pi(\beta) - X_i \right)$$
$$= \sum_{i=0}^n \sum_{j=0}^n \left(\pi(P, X) - \mathcal{L}_{ij} \right)$$

Non-Linear Least Squares: Gauss-Newton

$$E_{\text{NLS}}(\vec{x}) \equiv \frac{1}{2} \sum_i [f_i(\vec{x})]^2$$


All parameters
(all 3d points,
all cameras)
2D point index

$$E_{\text{NLS}}^0(\vec{x}) = \frac{1}{2} \sum_i (f_i(\vec{x}_0) + [\nabla f_i(\vec{x}_0)] \cdot (\vec{x} - \vec{x}_0))^2$$

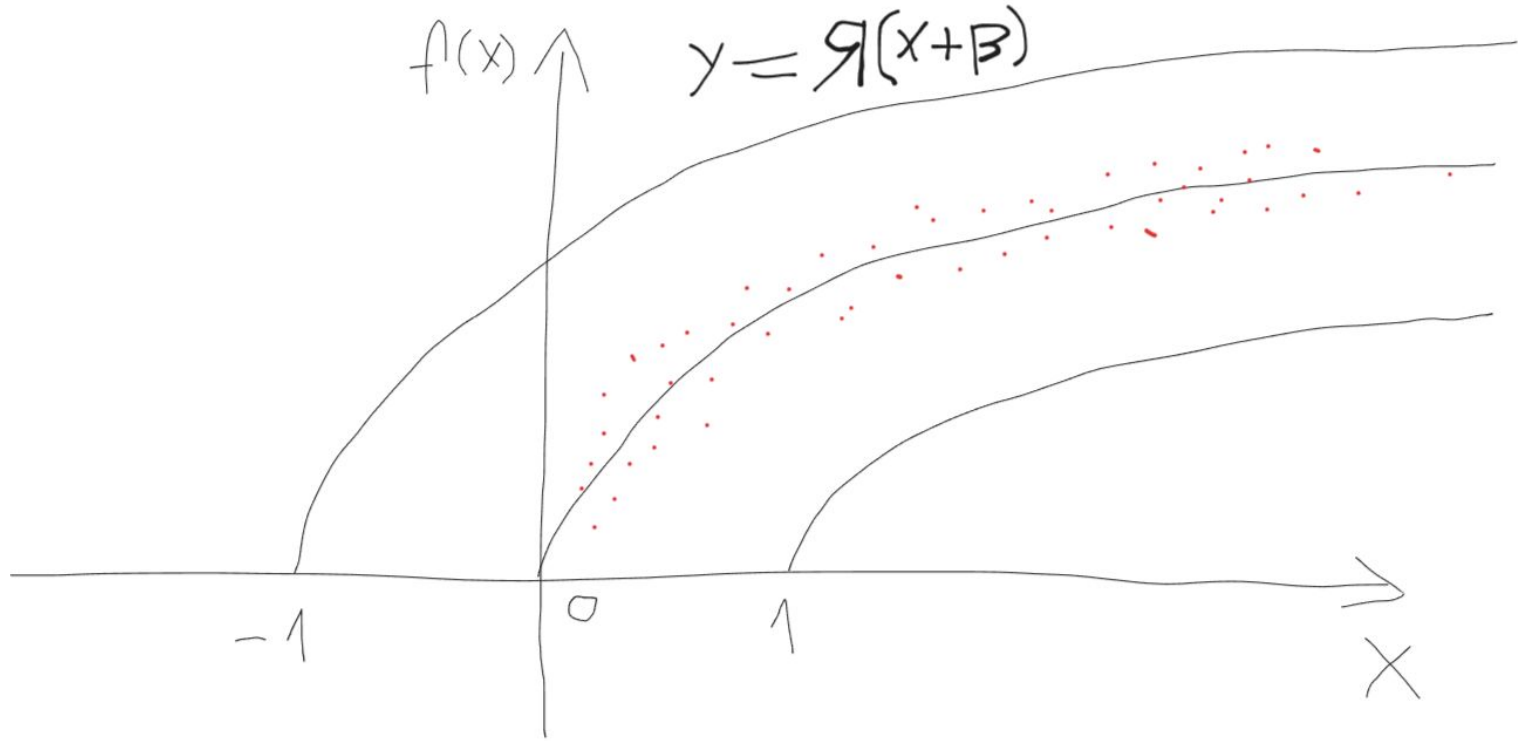
Non-Linear Least Squares: Gauss-Newton

step k (LLS): $\min_x \left\| \underset{-b}{F_k} + \underset{Ax}{J_k}(x - x_k) \right\|^2$

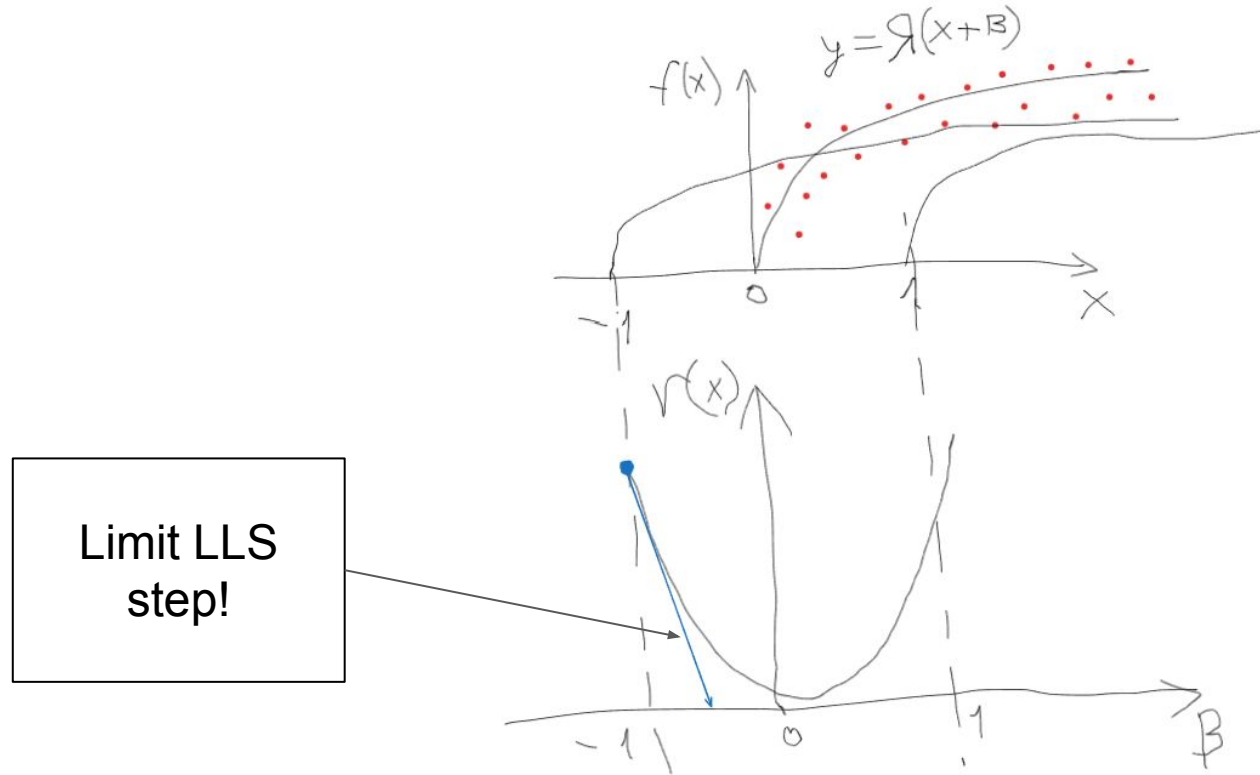
LLS formula reminder: $x = (A^T A)^{-1} A^T b$

step k update: $x_{k+1} = x_k - (J_k^T J_k)^{-1} J_k^T f(x_k)$

Non-Linear Least Squares: Overshooting Problem



Non-Linear Least Squares: Overshooting Problem



Non-Linear Least Squares: Levenberg-Marquardt

GN:
$$x_{k+1} = x_k - (J_k^T J_k)^{-1} J_k^T f(x_k)$$

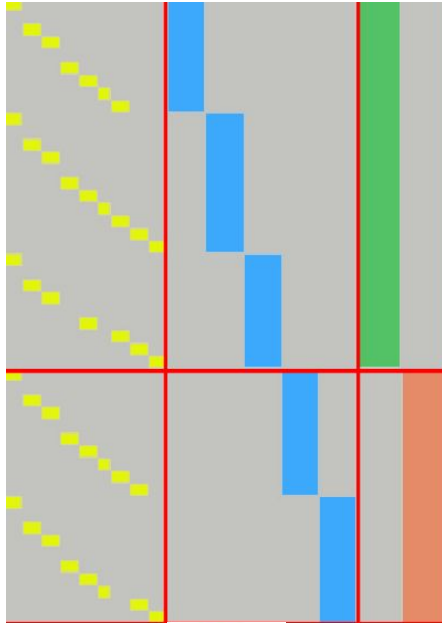
LM:
$$x_{k+1} = x_k - (J_k^T J_k + \lambda I)^{-1} J_k^T f(x_k)$$

Through constraints:
$$\min_x \sum_{i=1}^n \|f(x_i)\|^2 \quad \text{subject to} \quad \|x_{k+1} - x_k\| \leq \delta$$

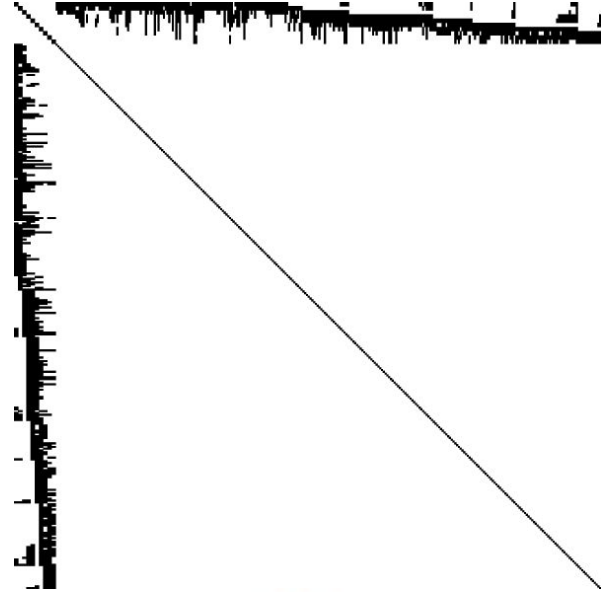
Non-Linear Least Squares: Levenberg-Marquardt

$$LM = GN + \lambda GD$$

Practical Considerations: Sparsity



J_k



$J_k^T J_k$

Practical Considerations: Implementation

[BA from Scipy Cookbook](#)

[Scipy Levenberg-Marquardt](#)

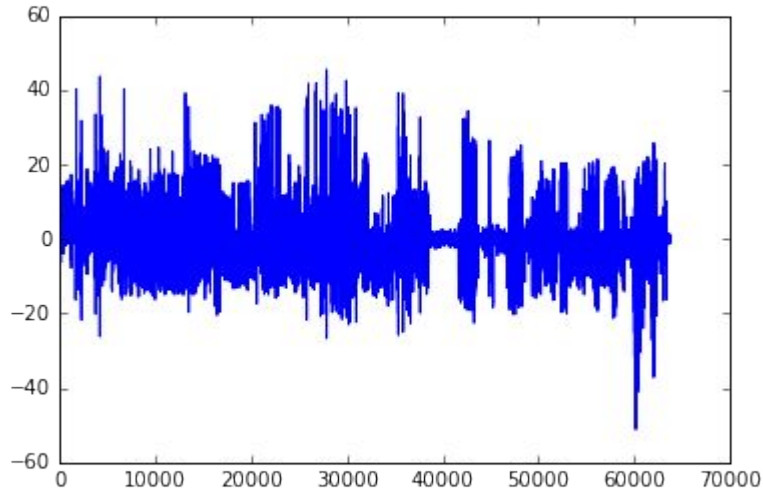
[JaxOpt Levenberg-Marquardt](#)

[Ceres NLLS](#)

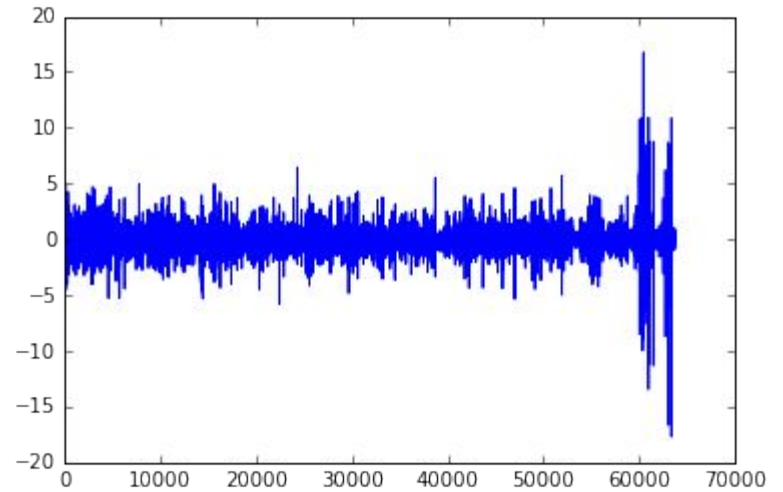
[Torch Levenberg-Marquardt](#)

My LM implementation (failed)

Bundle Adjustment: LM results (kinda)

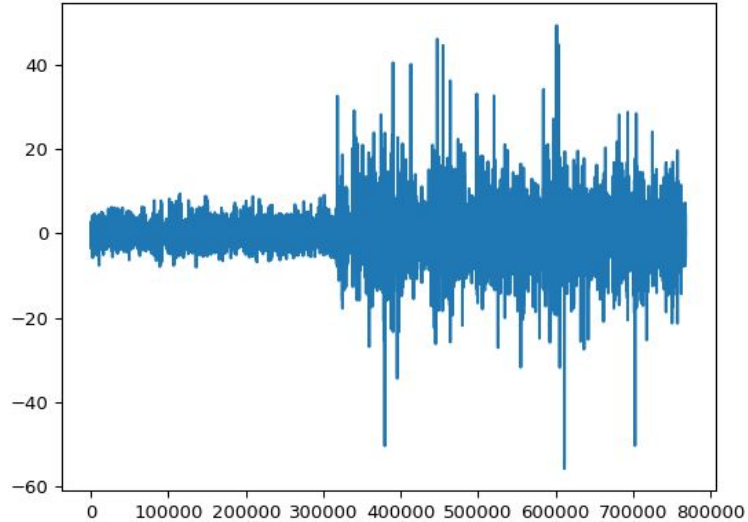


Before
Residuals

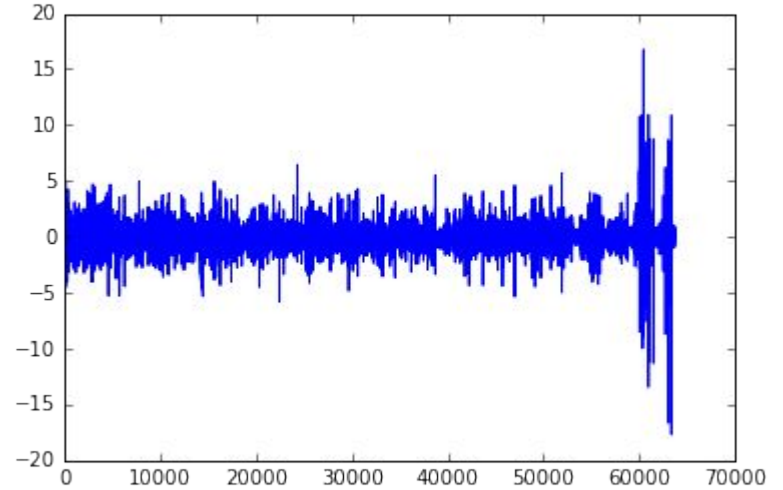


After
Residuals

Bundle Adjustment: Results Comparison

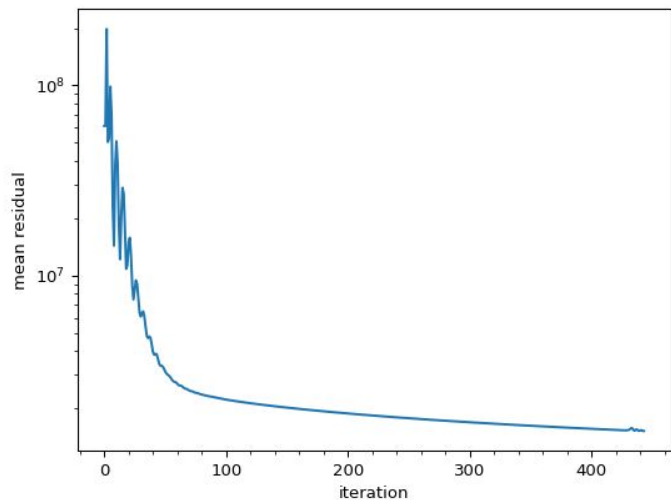


Adam
Residuals

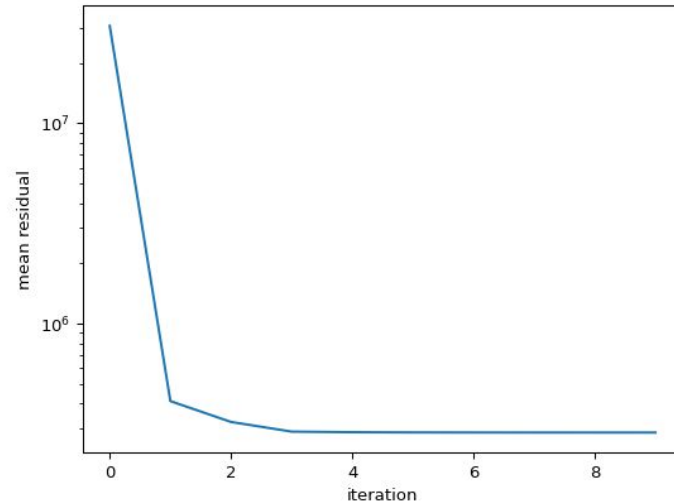


LM
Residuals

Bundle Adjustment: Results Comparison



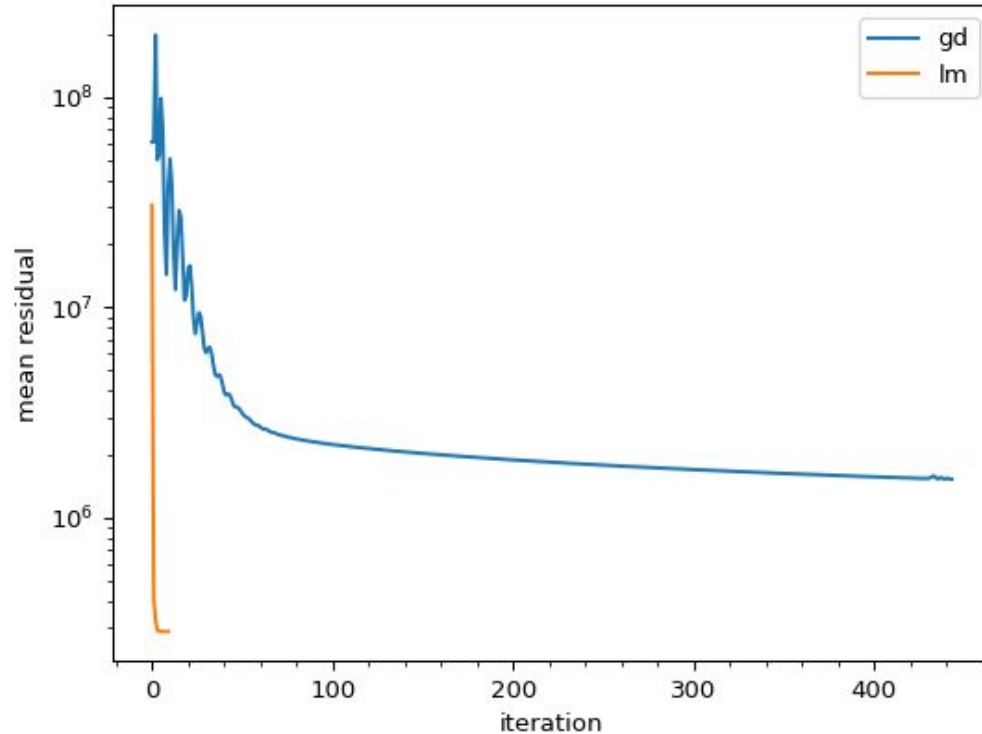
Adam Loss



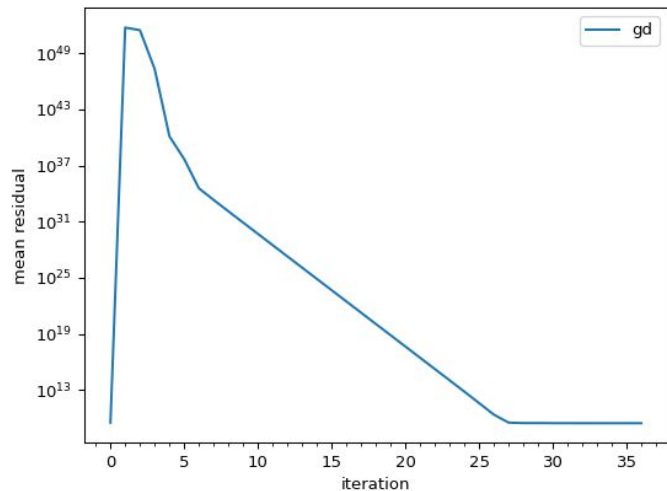
LM
Loss

Bundle Adjustment: Results Comparison

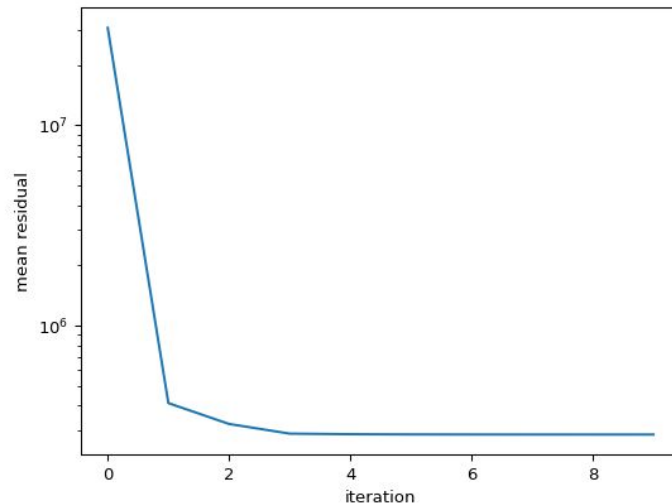
Adam



Bundle Adjustment: Results Comparison



My LM
Loss



LM
Loss

Practical Considerations: Tricks

Setting `scaling='jac'` was done to automatically scale the variables and equalize their influence on the cost function (clearly the camera parameters and coordinates of the points are very different entities). This option turned out to be crucial for successful bundle adjustment.

`x_scale` : *array_like or 'jac', optional*

Characteristic scale of each variable. Setting `x_scale` is equivalent to reformulating the problem in scaled variables `$xs = x / x\_scale$` . An alternative view is that the size of a trust region along `j`th dimension is proportional to `$x\_scale[j]$` . Improved convergence may be achieved by setting `x_scale` such that a step of a given size along any of the scaled variables has a similar effect on the cost function. If set to 'jac', the scale is iteratively updated using the inverse norms of the columns of the Jacobian matrix (as described in [\[JJMore\]](#)).

Practical Considerations: Tricks

LM:
$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\mathbf{J}_k^T \mathbf{J}_k + \lambda \mathbf{I})^{-1} \mathbf{J}_k^T f(\mathbf{x}_k)$$

Loss up, λ up
Loss down, λ down

Practical Considerations: Convergence

Factor	Gradient Descent	Gauss-Newton	Levenberg-Marquardt
Convergence Rate	Linear ($O(1/k)$) under convexity 1 7	Quadratic near solution with small residuals 2 6	Superlinear/quadratic (adaptive) 3 4 8
Residual Magnitude	Robust to large residuals (no assumptions) 1	Requires small residuals for fast convergence 2 6	Handles both small and large residuals via damping parameter 4 6 8
Curvature Conditions	Requires Lipschitz continuity of gradient 1 7	Requires full-rank Jacobian and mild nonlinearity 2 6	Works with rank-deficient Jacobian via regularization 3 4
Step Size Strategy	Fixed or backtracking line search 7	Often uses line search to ensure monotonic decrease 6	Adaptive damping parameter replaces explicit line search 4 8
Local Minimum Avoidance	Converges to nearest local minimum 1	May stagnate at saddle points without regularization 2	Improved local minimum escape via hybrid strategy 4 8
Computational Cost	Low per iteration (1st-order) 1	Moderate per iteration (Jacobian inversion) 6	Higher per iteration (matrix inversion with damping) 4 8
Robustness to Initialization	Sensitive; requires good initialization 1	Requires near-optimal initialization for fast convergence 2 6	More robust due to adaptive damping 3 4 8
Handling Singular Jacobians	Not applicable (doesn't use Jacobian)	Fails if Jacobian is rank-deficient 2	Regularizes Jacobian to handle rank deficiency 3 4

Conclusion

1. It's hard to get off of gradient descent/autodiff
2. ...